Autonomous Driving Project

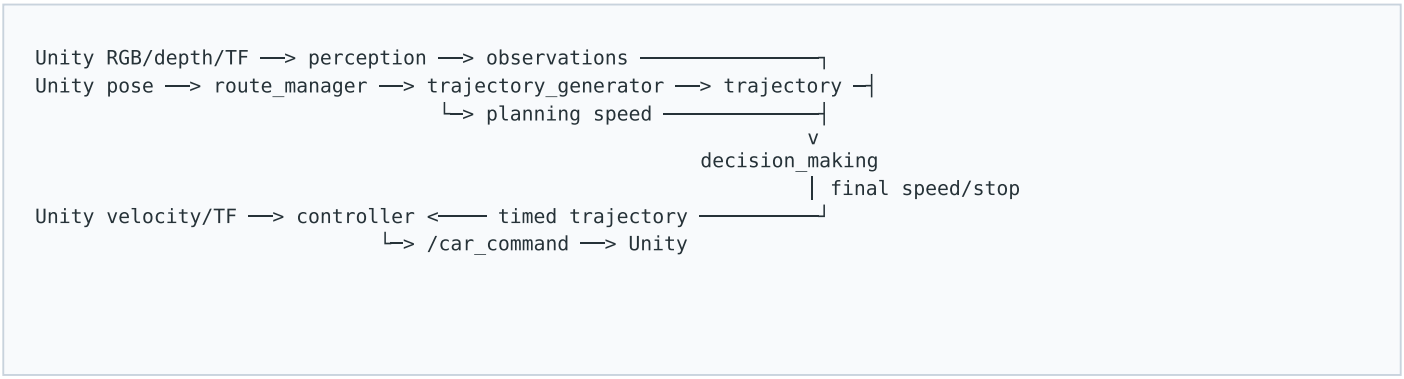
Standalone Documentation — Team 20

Lei Fang (03805710) | Yida Lyu (03816236) | Tao Zhang (03822999) | Jianing Huang (03814771)

ROS 2 Jazzy · Ubuntu 24.04 · Unity · 03 August 2026

Team 20 implemented a complete ROS 2 autonomous-driving pipeline for the supplied Unity urban course. The vehicle follows a validated 64-waypoint route, detects vehicles and traffic lights without the semantic camera, generates a dynamically feasible local trajectory, applies traceable safety decisions, and controls steering, throttle, and braking. One full_system.launch.py entry point starts the production graph; controller_node is the sole /car_command publisher.

Architecture and authoritative data flow



Main design choices

Area	Choice	Rationale / trade-off
Route	Fixed 64-waypoint YAML	Reproducible and inspectable on the fixed course; no global replanning.
Trajectory	Projection + Hermite/Bézier + dynamic envelope	Smooth path and per-point feasible speed/time under steering and acceleration limits.
Perception	YOLOv8n ONNX on RGB + depth/point cloud	Avoids semantic-camera shortcuts; route/HSV/temporal logic handles small lights.
Decision	Explicit safety-priority state machine	Traceable traffic-light, following, merge, emergency, and stale-input behavior.
Control	Pure Pursuit + PID + safe-stop gates	Transparent closed-loop control; stale or infeasible input produces braking.

Implemented requirement coverage

- Route completion; traffic-light response at waypoints 25/32/52/59; adaptive vehicle following and safe merge/emergency handling.
- Curvature-, lateral-acceleration-, acceleration-, and braking-constrained timed trajectory with a Team 20 ROS message.
- Production perception uses regular RGB, depth, calibration, and TF; segCamera.enabled is false.

ROS packages and nodes

Every workspace package and installed ROS node, including offline/non-production tools

Package	Node(s)	Functionality / ownership
simulation	JSON_param_reader; Unity_ROS_message_Rx; ROS_Unity_command_Tx; 5 static TF nodes	Course-supplied/reused Unity configuration, TCP sensors, UDP commands, and sensor TF.
perception	yolo_rgbd_detector_node	Tao: regular-RGB YOLO ONNX, RGB-D vehicle range, route-gated traffic-light observations.
perception	occupancy_map_node	Tao: filtered point clouds plus projected and extended occupancy maps.
external octomap_server	octomap_server	Reused OctoMap node producing 3-D octomap and projected-map topics.
route_manager	route_manager_node; waypoint_recorder (offline tool)	Lei: route validation/progress and local context; the offline recorder captures TF poses into route YAML.
autonomy_interfaces	no runtime node	Jianing: Team 20 TrajectoryPoint and PlannedTrajectory message definitions.
trajectory_generator	trajectory_generator_node	Jianing: projection, smoothing/resampling, curvature/dynamic checks, timed trajectory.
decision_making	obstacle_monitor_node	Yida: planning-corridor point cloud plus stable route-gated RGB-D vehicle range fusion.
decision_making	traffic_light_state_machine_node	Yida: safety-priority traffic-light/following/merge/emergency state machine and final intent.
controller	controller_node	Yida: input validation, Pure Pursuit, PID, bounded commands, safe stop, sole /car_command.
system_bringup	launch; live_visualization; 2 RViz nodes	Shared integration: one production launch. Visualization is diagnostic only.
dummy_controller	dummy_controller_node (not production)	Course-supplied example. It must never run with Team 20 controller_node.

Interface contract

The custom /planning/trajectory contains ordered pose, signed curvature, target speed, and relative time. /planning/path remains the geometric interface for RViz and obstacle corridors. /planning/target_speed is a proposal: decision_making publishes the final /decision/target_speed and stop request used by the controller. Team 20 authored TrajectoryPoint.msg and PlannedTrajectory.msg, satisfying the required own-message element.

Functional modules and core equations

How sensing is converted into one safe, dynamically feasible vehicle command

Perception and mapping

Tao Zhang

The regular RGB stream feeds YOLOv8n; aligned depth and calibration turn detections into metric vehicle ranges. Route-gated image evidence and temporal filtering classify small traffic lights. Point clouds produce local occupancy maps.

RGB-D BACK-PROJECTION

$$X = (u - c_x)Z/f_x, \quad Y = (v - c_y)Z/f_y$$

Route management

Lei Fang

A validated 64-waypoint YAML route supplies progress, local path, target point, speed limit, and intersection semantics. Ego pose is projected onto the route so downstream modules share one longitudinal reference.

KEY OUTPUTS

Waypoint progress · local path · route context · speed limit

Trajectory generation

Jianing Huang

The local route is smoothed and resampled, then annotated with curvature, speed, and relative time. Forward/backward passes enforce steering, lateral acceleration, acceleration, and braking bounds before publication.

DYNAMIC SPEED ENVELOPE

$$v_{cap} = \min(v_{route}, \sqrt{a_{lat,max}/|\kappa|})$$

Decision making

Yida Lyu

A safety-priority state machine combines traffic-light state, obstacle range, merge context, and stale-input checks. It can follow a lead vehicle, stop before a controlled waypoint, or request an emergency stop.

KEY OUTPUTS

Final target speed · stop request · named behaviour state

Vehicle control

Yida Lyu

Pure Pursuit computes steering from a look-ahead target; PID regulates longitudinal speed. Bounds, watchdogs, and feasibility gates convert the final target into the only production /car_command stream.

PURE PURSUIT STEERING

$$\delta = \tan^{-1}(2L\sin\alpha/l_d)$$

System bringup and safety

Shared integration

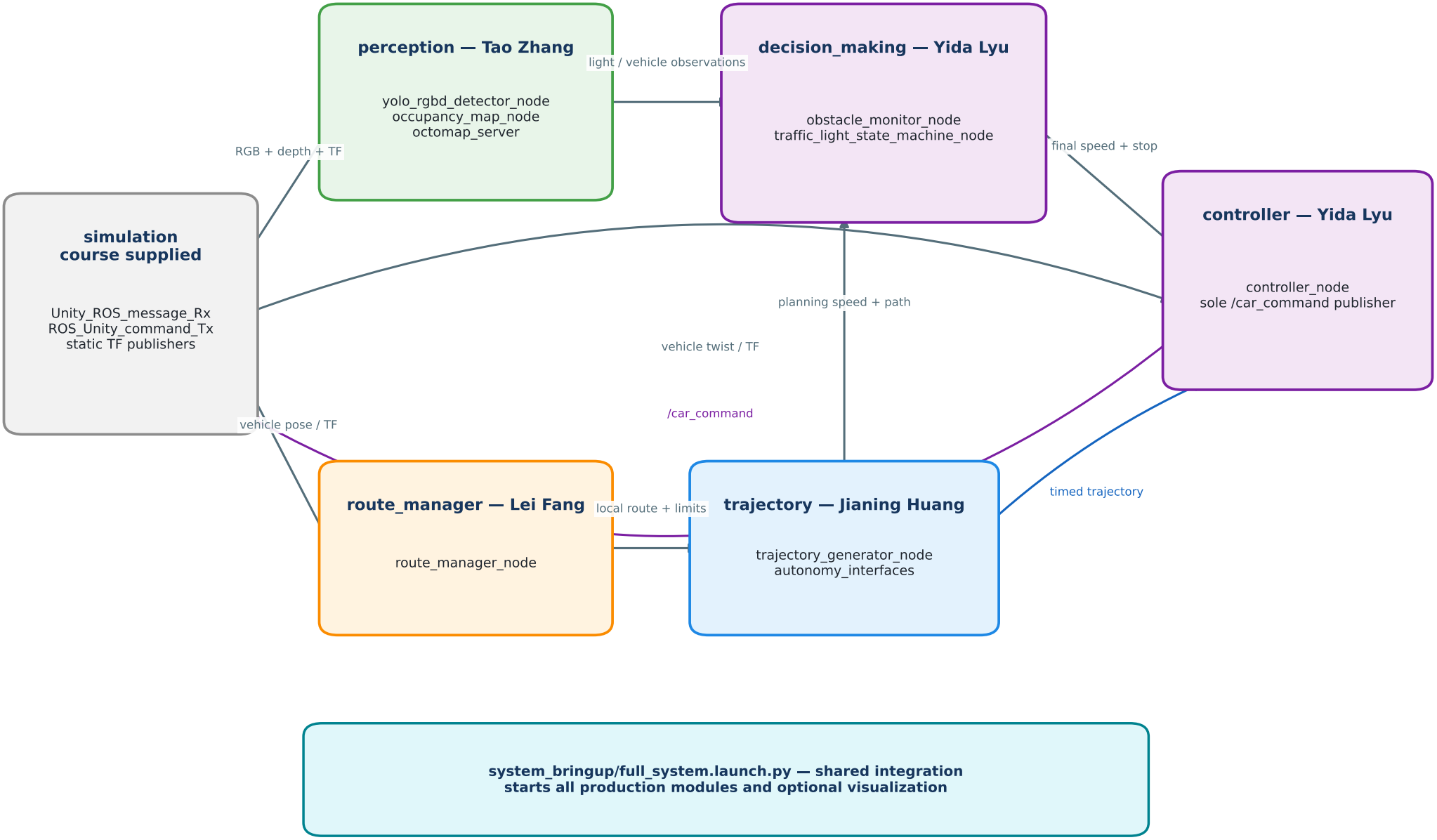
One launch file starts simulation, TF, perception, route, trajectory, decision, controller, and optional visualization. Motion is disabled by default; stale or invalid inputs lead to braking instead of an unchecked command.

OPERATIONAL BOUNDARY

Single launch entry point · motion off by default · fail-safe braking

Runtime ROS 2 graph and contribution ownership

Readable vector overview; the complete topic-level graph is linked below



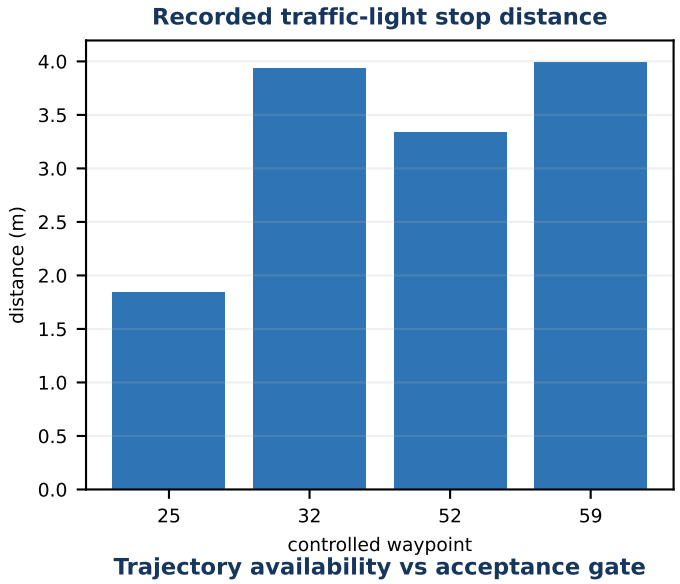
Open the complete full-resolution ros_graph.svg (all nodes and topics)

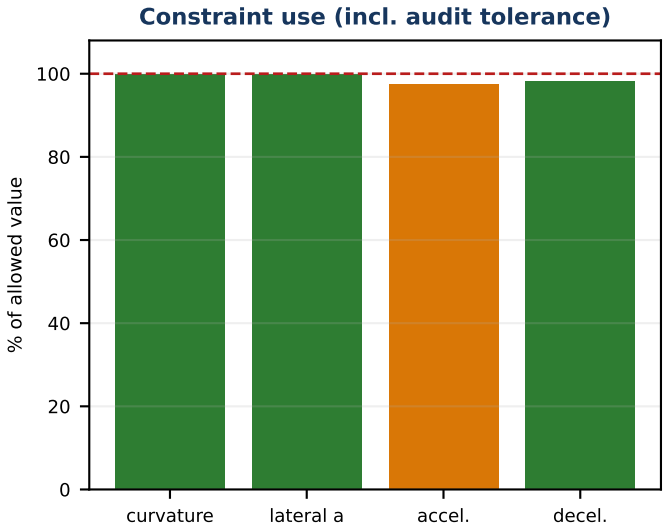
Repository path: project/docs/ros_graph.svg

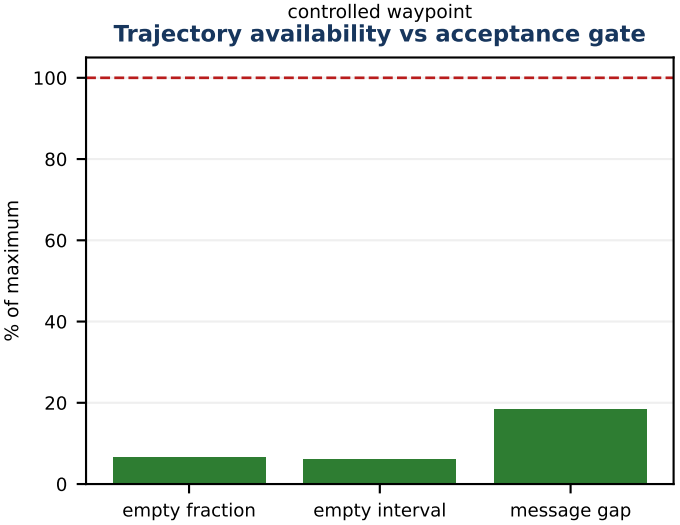
Recorded results and known limitations

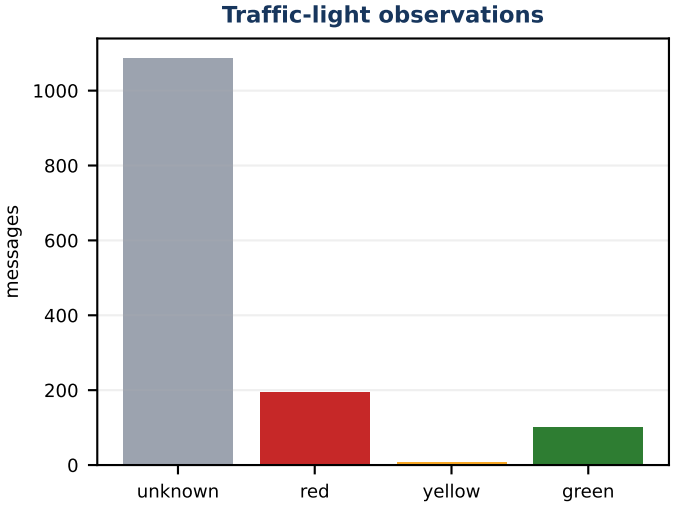
Retained production run: 01 August 2026 · one reproducible evidence example

Waypoint 63 reached · mission time 210.231 s · 2,115 trajectories / 70,008 points · trajectory audit passed · tracking RMSE 0.668 m · no runtime process death







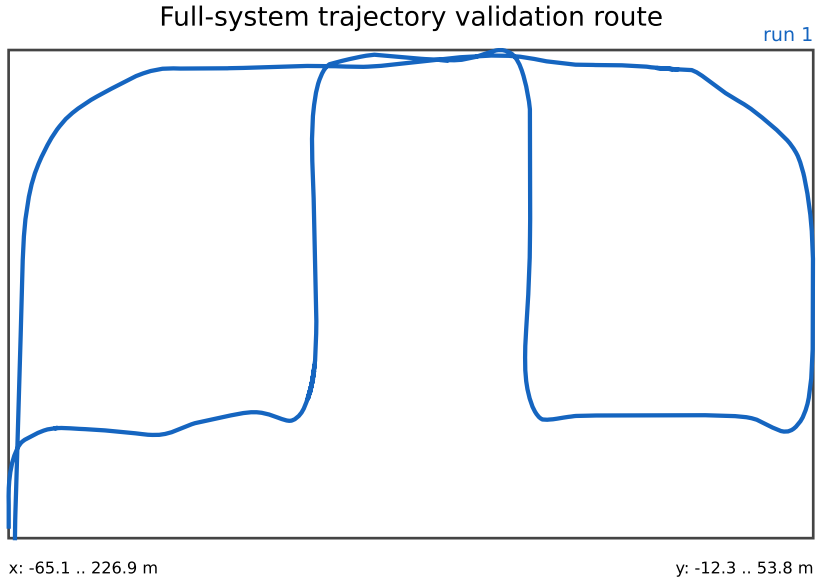


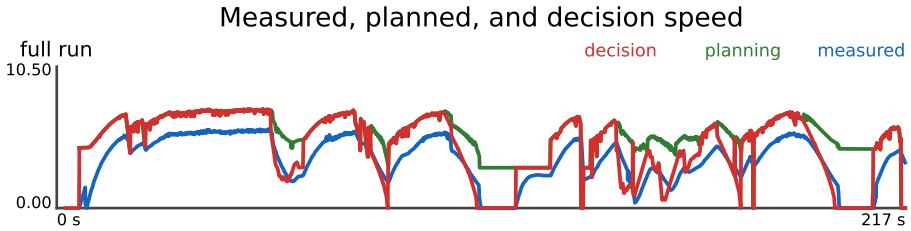
Known limitations / incomplete extensions

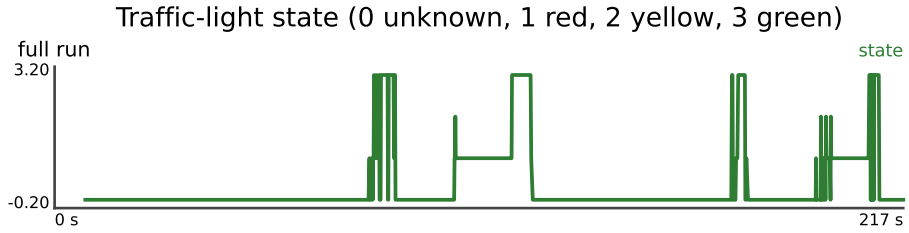
- The bundled detector is COCO-pretrained YOLOv8n, not a Team 20 Unity-fine-tuned model. Small lights need route gates, per-intersection ROIs, HSV/layout evidence, and temporal filtering.
- A changed map, route, camera resolution, or field of view requires recalibration and full validation; the fixed YAML route does not globally replan.
- The trajectory envelope omits jerk, tyre-force coupling, steering rate, drivetrain lag, and free-space obstacle replanning. Obstacle risk is handled longitudinally.
- Missing/stale sensor data deliberately brakes; brief TCP/depth interruptions may therefore cause conservative transient stops.
- Large raw bags are retained through the documented Git LFS reproduction record; a separate no-event run would strengthen comparative evidence.

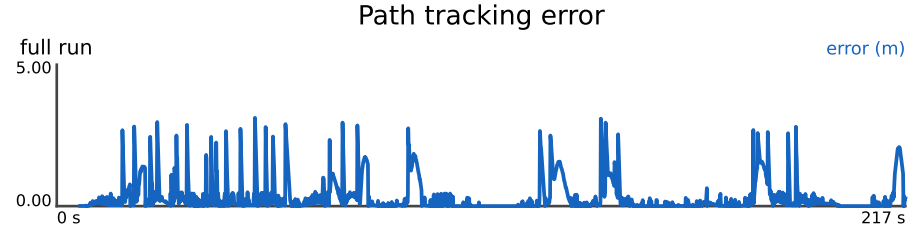
Mission-result evidence

Retained full-system run; source SVGs are redrawn as native vectors for lossless zoom









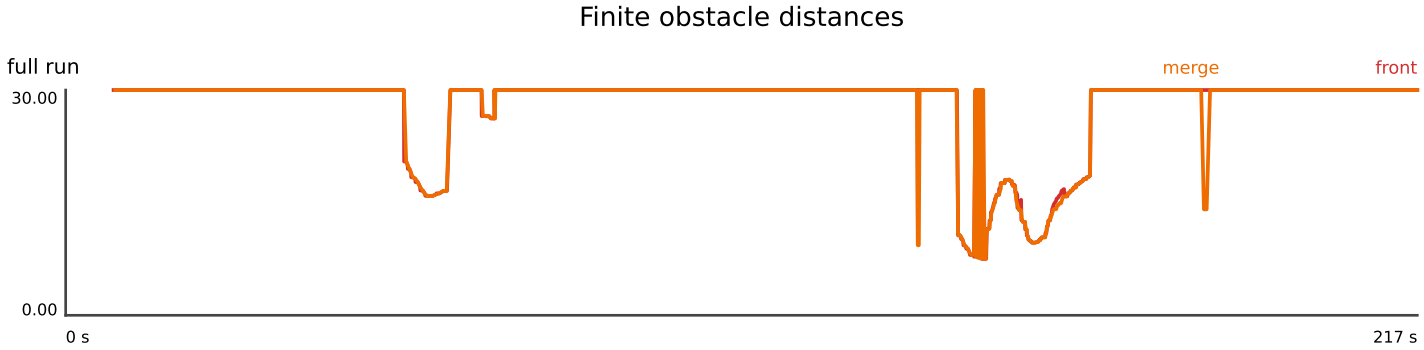
Source SVGs: route_trace · speed_profile · tracking_error · traffic_light_state

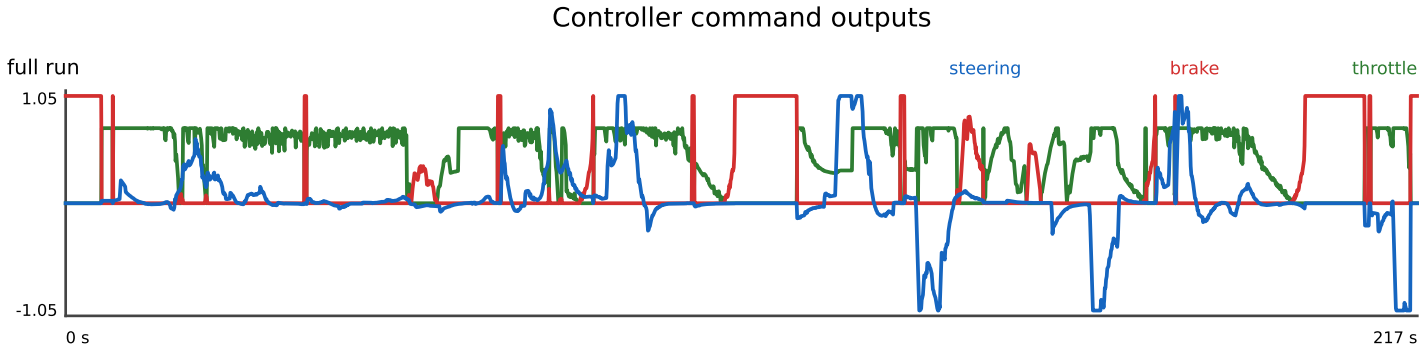
Directory: project/mission_results/ · Numerical evidence: metrics.json and validation_provenance.json

Analysis — the vehicle reached waypoint 63 in 210.2 s. The speed trace reflects planned slowdowns and stops rather than a constant-speed replay. Tracking RMSE was 0.668 m; the bounded error supports closed-loop path following. Traffic-light transitions contain all four states and the recorded red-light stops occurred before their controlled waypoints.

Obstacle and actuator safety evidence

Full-run signals retained alongside the route, speed, tracking, and traffic-light plots





Analysis — finite ranges indicate intervals in which the obstacle monitor had a relevant front or merge target; the Event2 minimum was 7.502 m. The vehicle completed the route without collision or emergency-state entry. The controller trace shows bounded steering and alternating acceleration/braking phases. No retained sample commanded throttle and brake simultaneously, which confirms the mutual-exclusion gate at the final command boundary.

Team contribution, reused code, and bibliography

Ownership describes project implementation, not invention of public algorithms

Member	Primary responsibility	Traceable work
Lei Fang 03805710	route_manager; integration	Route YAML/model, validation, waypoint progress, local route/context, speed limits, tests.
Yida Lyu 03816236	decision_making; controller; integration	Obstacle fusion, behavior state machine, Pure Pursuit/PID, safety and command interfaces.
Tao Zhang 03822999	perception; integration	YOLO RGB-D ROS integration, light processing, vehicle range, point clouds/maps.
Jianing Huang 03814771	trajectory_generator; autonomy_interfaces; integration; documentation	Projection, smoothing/resampling, constrained time profile, custom messages, documentation.

All four members contributed to interface/configuration alignment, system_bringup, end-to-end tests, and dynamic validation at their module boundaries. No finer individual split of shared integration work is claimed.

External code and reuse boundary

- Course-supplied: Unity simulator, original simulation/ROS bridge, and dummy_controller. These are retained/reused and not claimed as Team 20 code.
- Third-party: YOLOv8n/COCO weights and Ultralytics tooling, ONNX Runtime, OpenCV, ROS 2/rclcpp/rclpy, TF2, OctoMap, Eigen, NumPy, PyYAML, nlohmann/json, and the Unity runtime. Their licences remain applicable.
- Public methods: Pure Pursuit, PID, Hermite/Bézier interpolation, constant-acceleration reachability, and finite-state machines. Team 20 owns its project implementations, ROS integration, tests, and tuning, not the underlying methods.
- Model record: yolov8n.onnx, 640×640, 80 COCO classes, SHA-256 8707bb120b03a6bf8a199e7c35c6d47ad5f83de4fdd233110db270f7d355a3be.

Bibliography

1. Technical University of Munich, Introduction to ROS: Autonomous Driving Project Assignment, AutonomousDriving.pdf, 2026.

2. R. C. Coulter, Implementation of the Pure Pursuit Path Tracking Algorithm, CMU-RI-TR-92-01, Carnegie Mellon University, 1992.

3. G. Farin, Curves and Surfaces for CAGD: A Practical Guide, 5th ed., Morgan Kaufmann, 2002.

4. K. J. Åström and T. Hägglund, PID Controllers: Theory, Design, and Tuning, 2nd ed., ISA, 1995.

5. Open Robotics, ROS 2 Jazzy documentation, https://docs.ros.org/en/jazzy/, accessed 2026-08-01.

6. Ultralytics, Model Export with Ultralytics YOLO, https://docs.ultralytics.com/modes/export/, accessed 2026-08-01.

7. Microsoft, ONNX Runtime C++ documentation, https://onnxruntime.ai/docs/get-started/with-cpp.html, accessed 2026-08-01.

8. A. Hornung et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” Autonomous Robots 34, 189–206, 2013.

Formal source: DocumentationTeam20.md · ROS graph: project/docs/ros_graph.svg · Validation data: project/mission_results/metrics.json · Reproduction record: project/Rosbag_reproduction/README.md